

Astrostats

for people in a hurry

Kyle Gersbach

Based on presentations by Neil Cornish, Steve Taylor, Nima Laal, and many others

Outline

- What is a statistic?
- Philosophy of stats
 - Frequentist vs Bayesian
- Basic probabilities
- Bayesian example
- What is data?
- What is an MCMC?

But what is a statistic?

A function which turns measurements into a value

- Almost anything is a statistic

Statistics should be **useful**

- Makes a prediction or is descriptive
- Often needs to be calibrated

$$avg(x) = \frac{1}{n} \sum_i^N x_i$$

$$med(x) = x_{(n+1)/2}$$

$$f(x) = \sum_i^N 4x_i^3 + x_i^2$$

Frequentist

Probabilities of data

- Data is probabilistic
- Model is fixed

Bayesian

Probabilities of model

- Data is fixed
- Model is probabilistic
- Updating prior information

A coin tosses to a statistician: Frequentist

- Assuming a fair coin

$$p(H) = 0.5$$

$$p(T) = 1 - p(H)$$

- Assuming independent probabilities

$$p(H|H) = 0.5 \quad p(H|T) = 0.5$$

$$p(HH) = p(H) * p(H|H) = p(H)^2$$

Bayes Theorem

$$p(A) = p(A|B)p(B)$$

Bayes Theorem

$$p(A) = p(A|B)p(B)$$

$$p(A|B)P(B) = p(B|A)p(A)$$

Bayes Theorem

$$p(A) = p(A|B)p(B)$$

$$p(A|B)p(B) = p(B|A)p(A)$$

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

Bayes Theorem

$$p(A) = p(A|B)p(B)$$

$$p(A|B)p(B) = p(B|A)p(A)$$

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

$$p(\text{model}|\text{data}) = \frac{p(\text{data}|\text{model})p(\text{model})}{p(\text{data})}$$

Bayes Theorem

$$p(A) = p(A|B)p(B)$$

$$p(A|B)p(B) = p(B|A)p(A)$$

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

Posterior

Likelihood

Prior

$$p(\text{model}|\text{data}) = \frac{p(\text{data}|\text{model})p(\text{model})}{p(\text{data})}$$

Evidence

A coin tosses to a statistician: Bayesian

A new question to ask: Is the coin really fair?

- Prior: the coin is most likely fair $p(\text{fair})$
- How confident are we that the coin is fair after flipping heads?

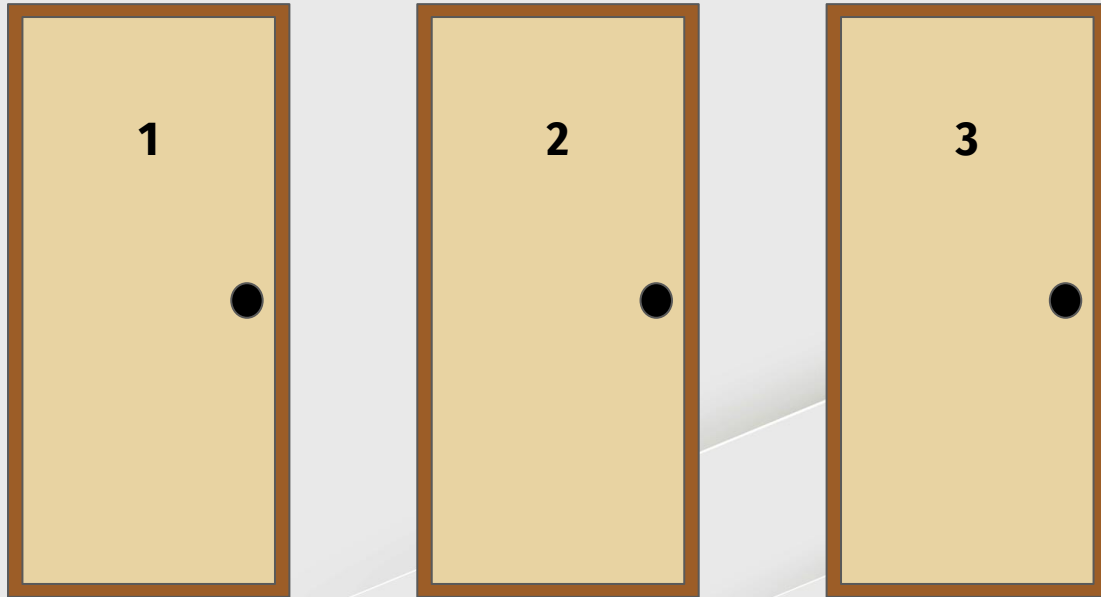
$$p(\text{fair}|H) = \frac{p(H|\text{fair})p(\text{fair})}{p(H)}$$

$$p(H) = p(H|\text{fair})p(\text{fair}) + p(H|\text{biased})p(\text{biased})$$

The Monty Hall problem

3 doors, 2 have goats, 1 has a car

Pick a door



The Monty Hall problem

3 doors, 2 have goats, 1 has a car

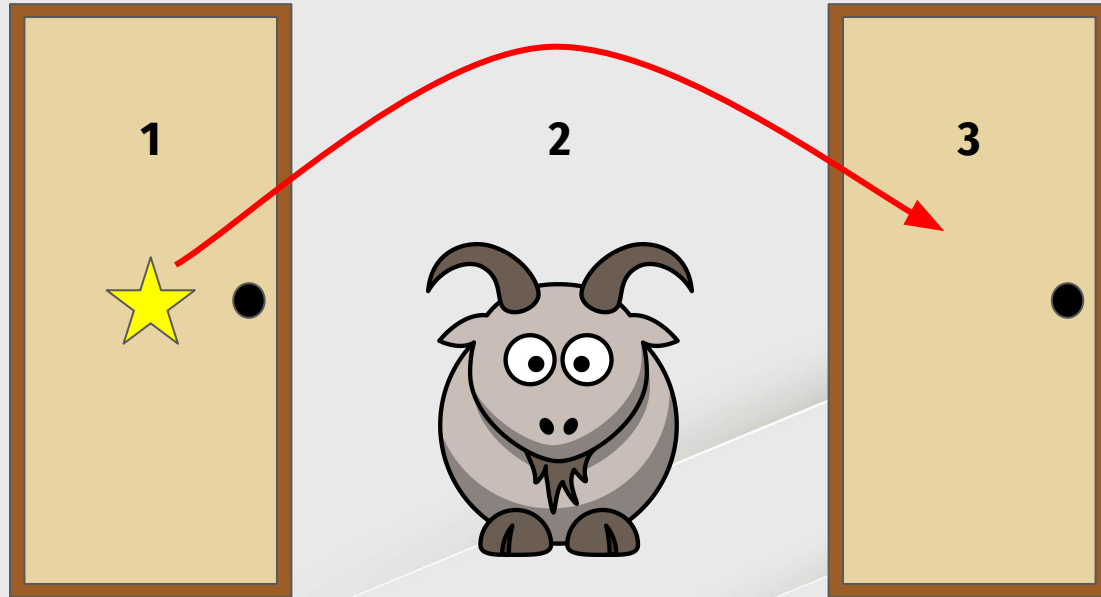
Pick a door -> Reveal one goat



The Monty Hall problem

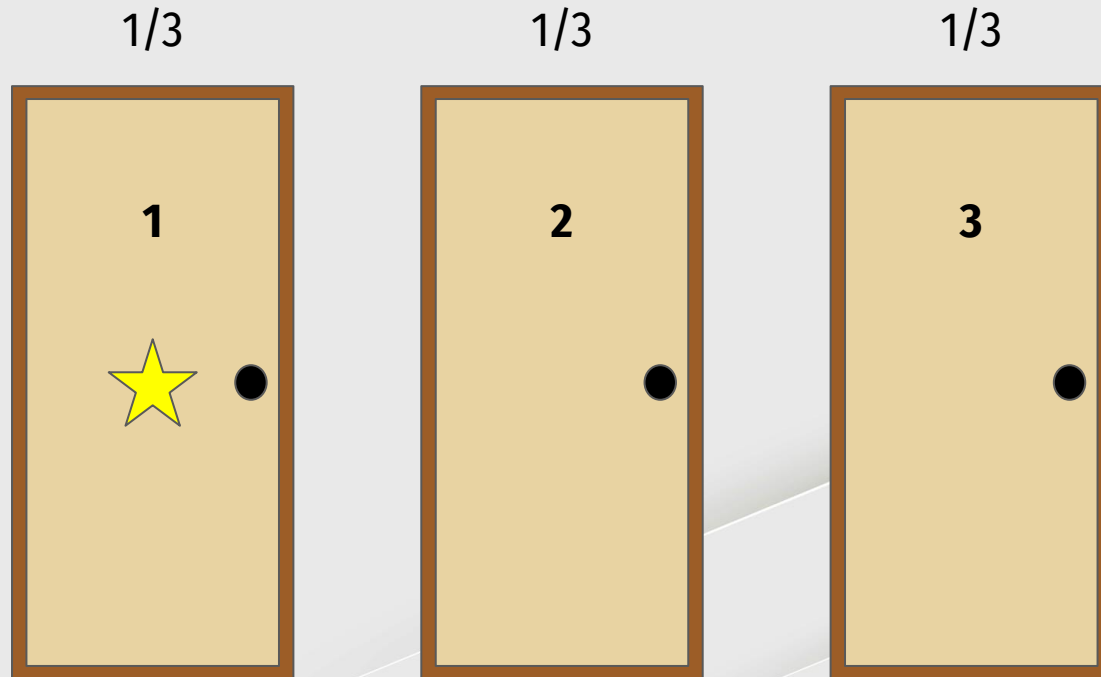
3 doors, 2 have goats, 1 has a car

Pick a door -> Reveal one goat -> Should you switch?



The Monty Hall problem

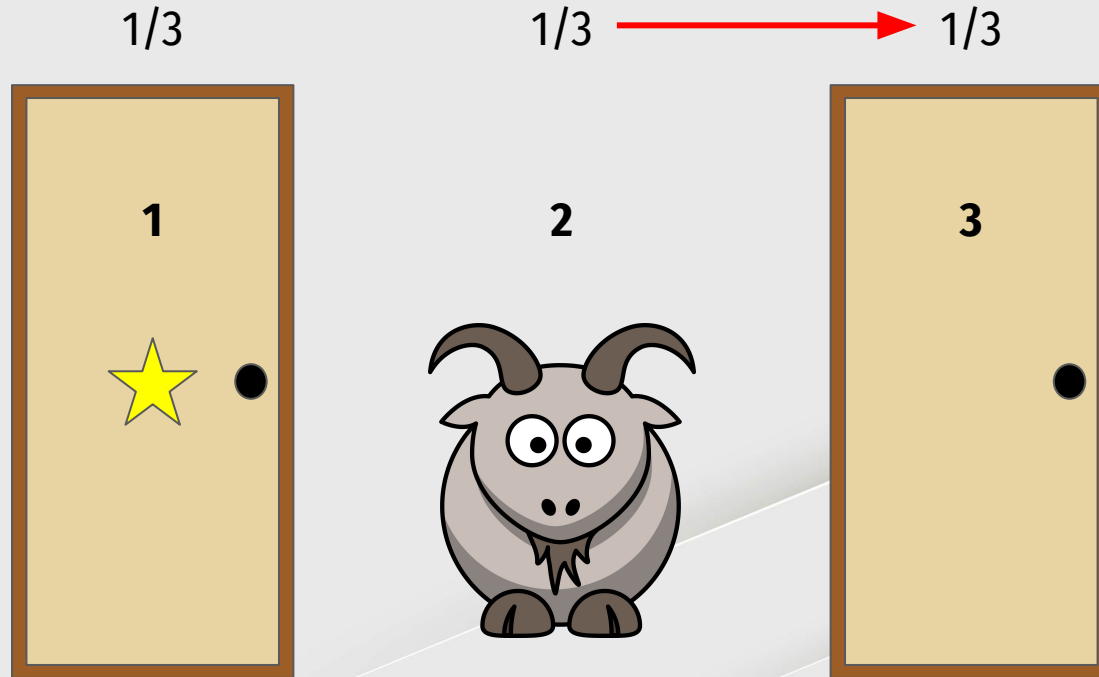
3 doors, 2 have goats, 1 has a car - Should you switch?



The Monty Hall problem

3 doors, 2 have goats, 1 has a car - Should you switch?

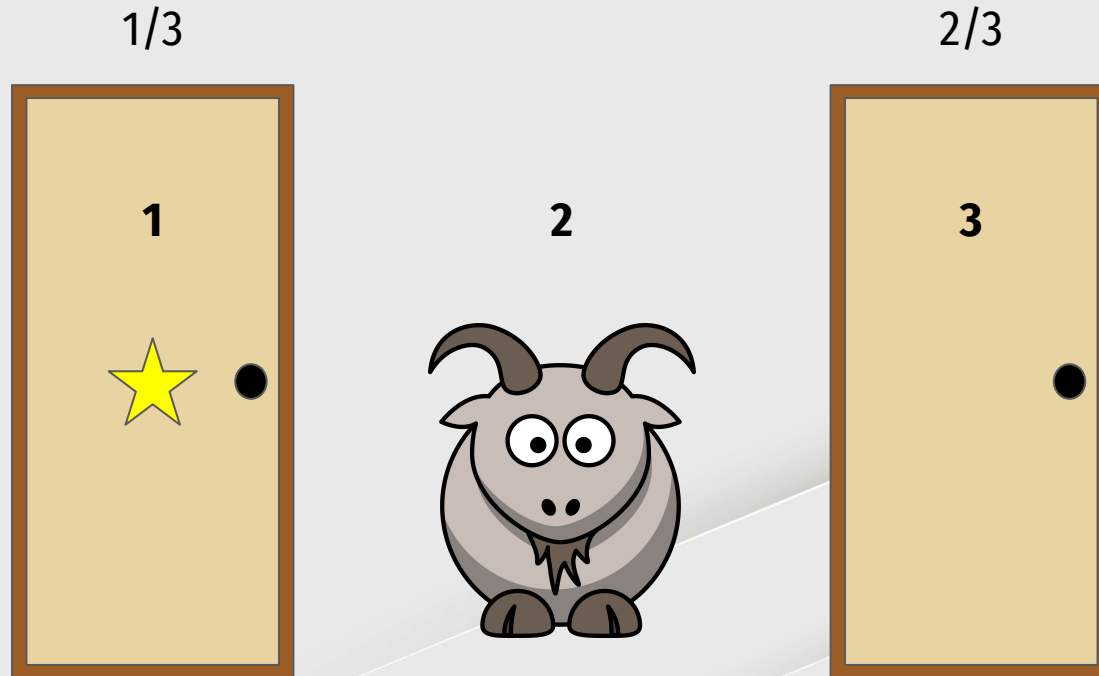
Conservation of probability



The Monty Hall problem

3 doors, 2 have goats, 1 has a car - Should you switch?

Conservation of probability



The Monty Hall problem

Let's do a Monte Carlo simulation!

```
1  def monte_hall(ndoors=3):
2      doors = [0,0,0] # Make doors, 0 = goat, 1 = car
3      doors[np.random.randint(ndoors)] = 1 # Randomly place car behind one door
4      choice = 0 # Contestant chooses door 0
5      doors.pop( doors[1:].index(0) + 1 ) # Host opens a door with a goat
6      switch = doors[1] # Contestant's switch choice
7      return switch
8
9  wins = 0
10 n_trials = 10_000
11 for i in range(n_trials):
12     wins += monte_hall()
13
14 print(f"Contestant wins {wins} times out of {n_trials} games, {wins/n_trials*100}% win rate.")
```

✓ 0.1s

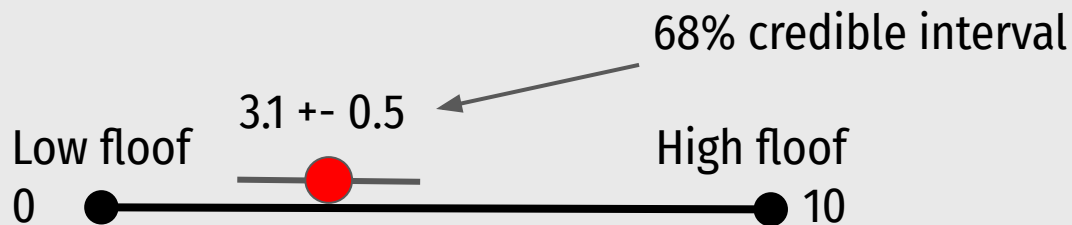
Contestant wins 6670 times out of 10000 games, 66.7% win rate.

Side note: Knowledge of programming often lets you simulate answers before mathing it!

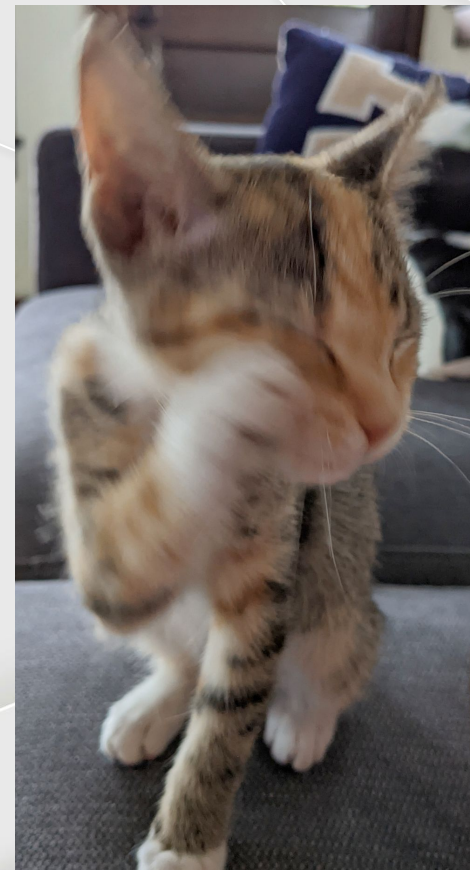
Lets make our own Monte Carlo sampler!

Suppose we have a kitten, Callisto.

Callisto is a low to moderately floofy kitten



3.1 $\pm$ 0.5



My cat, Callisto!

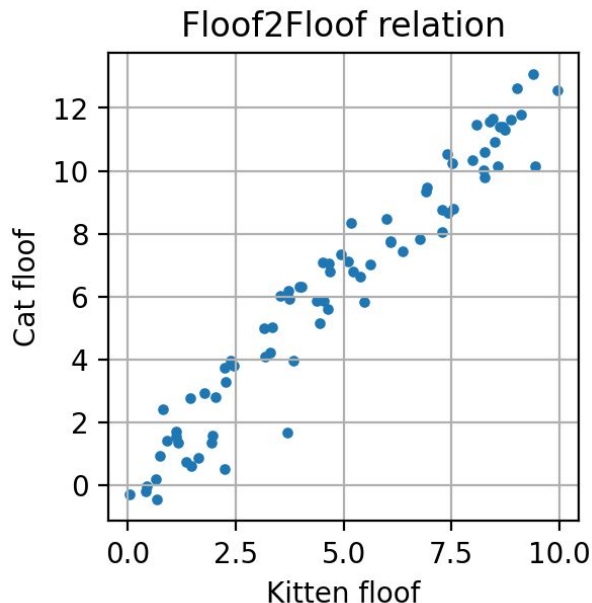
Floof2Floof relationship

There is a kitten floof to cat floof relation (floof2floof)

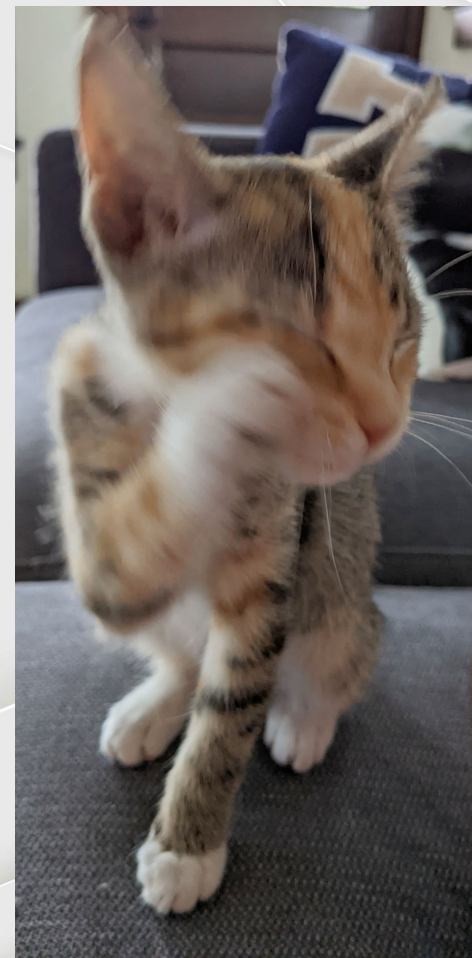
K = Kitten Floofiness

C = Cat Floofiness

$$C = 1.3 * K \pm 1$$

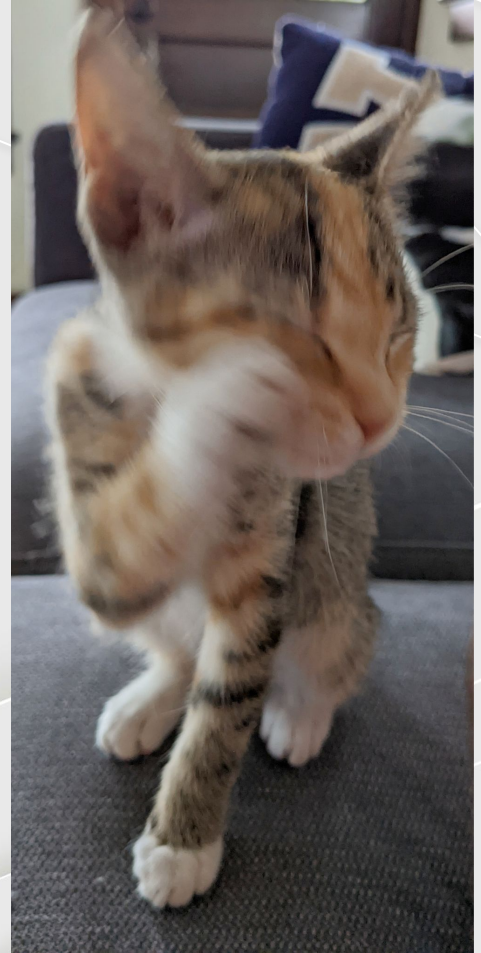


3.1 $\pm$ 0.5



What will Callisto's adult floof be?

3.1 $\pm$ 0.5

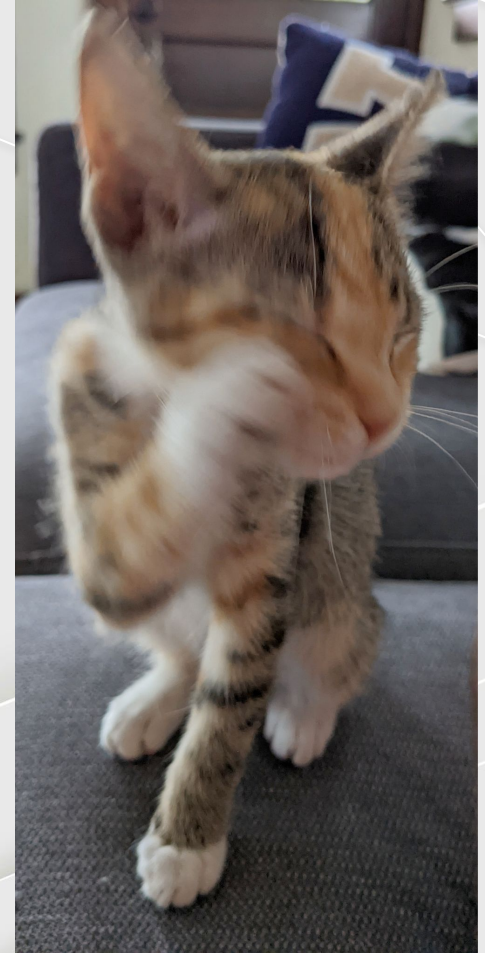


What will Callisto's adult floof be?

“Just put the values into the equation”

- Simple but Deceptive
- Does not account for uncertainty in measurement!
- No uncertainty in relationship either!

3.1 $\pm$ 0.5



What will Callisto's adult floof be?

“Just put the values into the equation”

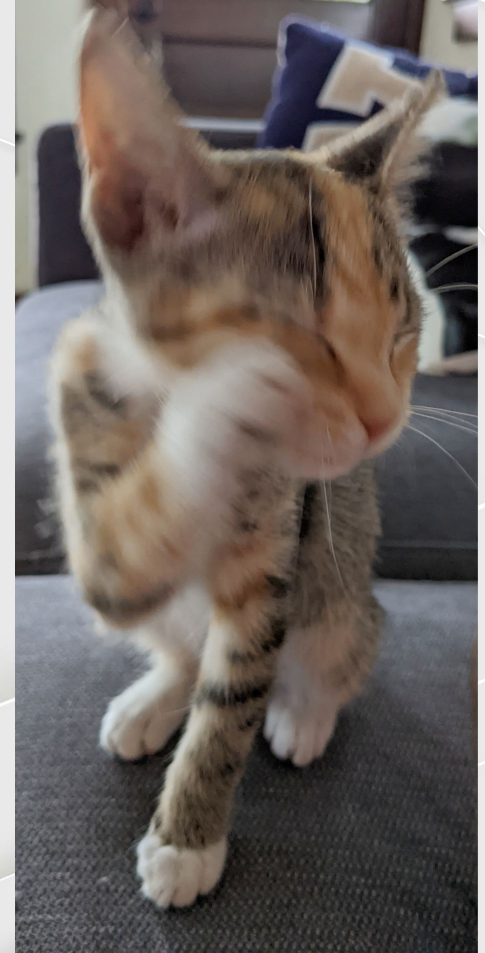
- Simple but Deceptive
- Does not account for uncertainty in measurement!
- No uncertainty in relationship either!

Lets use a Monte carlo simulation!

- Distribution on kitten floofiness
- Distribution for floof2floof error

Find a distribution on the final cat floofiness!

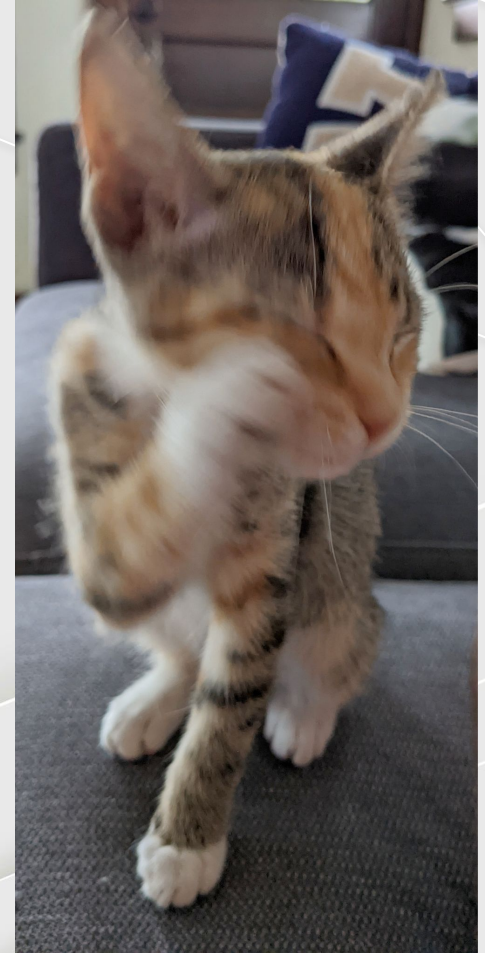
3.1 $\pm$ 0.5



What is the distribution on Callisto's adult floof?

3.1 $\pm$ 0.5

What is the initial distribution of Kitten floof values?

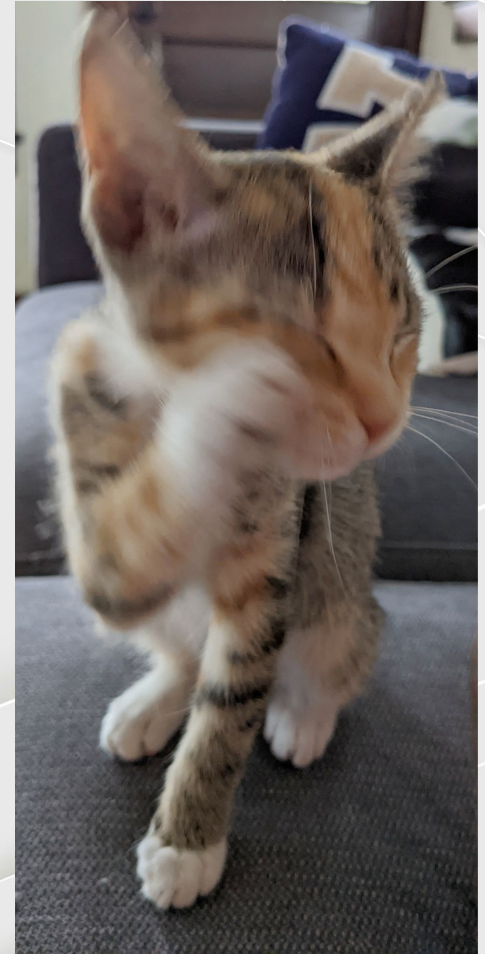
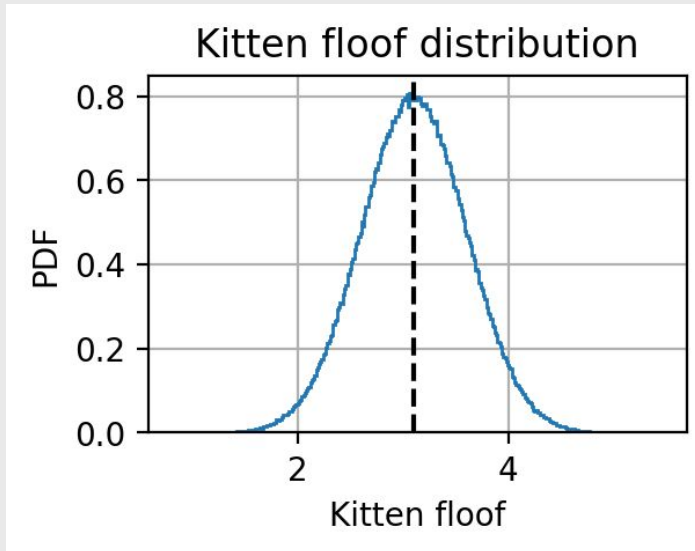


What is the distribution on Callisto's adult floof?

3.1 $\pm$ 0.5

What is the initial distribution of Kitten floof values?

$$\mathcal{N}(\mu = 3.1, \sigma = 0.5)$$

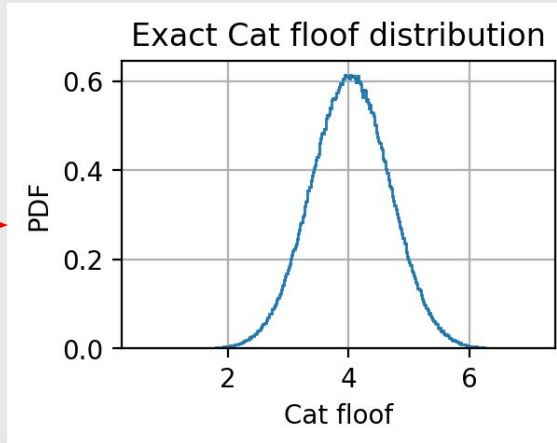
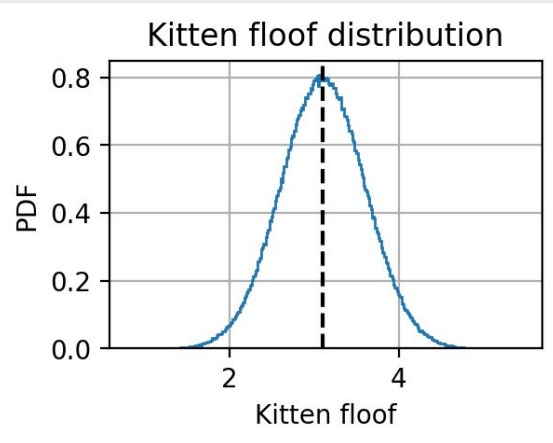


Put the distribution through the relation

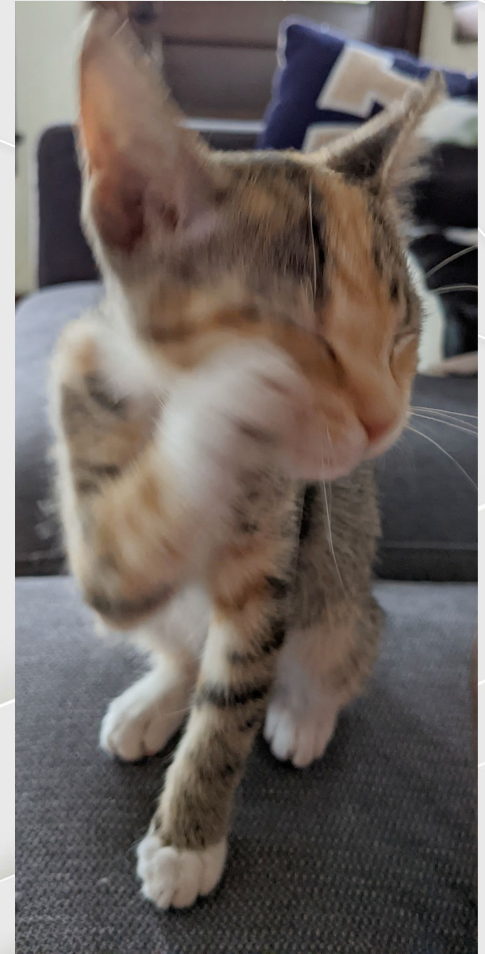
K = Kitten Floofiness

C = Cat Floofiness

$$C = 1.3 * K$$



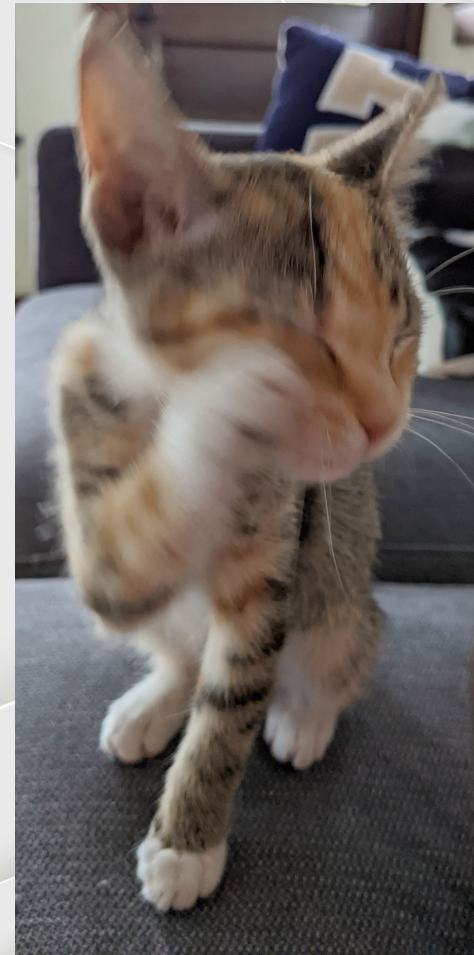
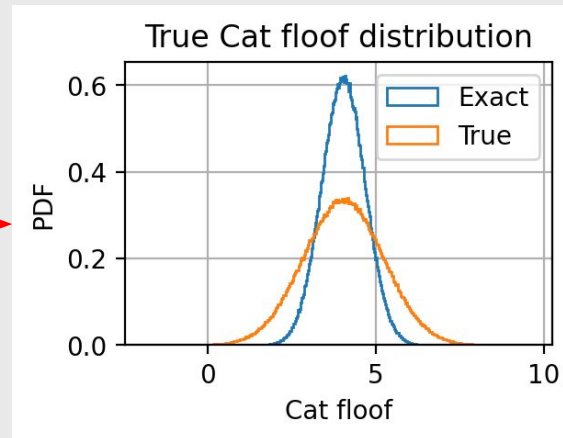
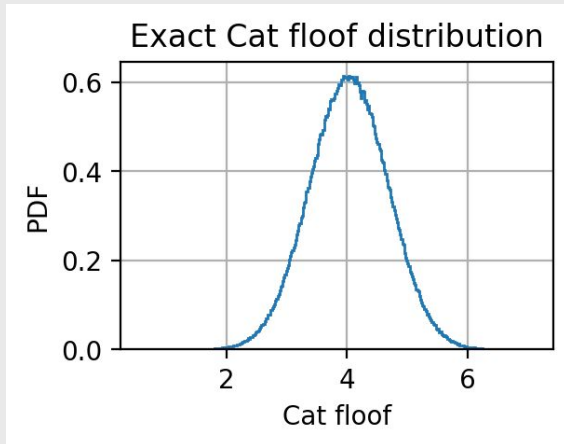
3.1 $\pm$ 0.5



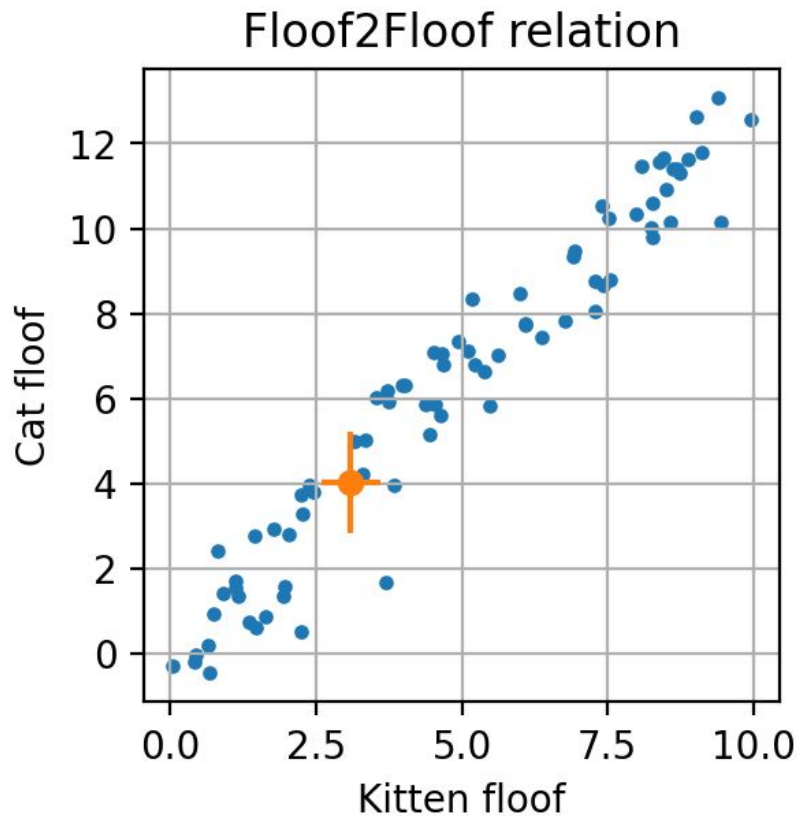
**Put the distribution through the relation
+ Add Noise in the relation!**

$$C = 1.3 * K + \mathcal{N}(\mu = 0, \sigma = 1)$$

3.1 $\pm$ 0.5



Plot our results!

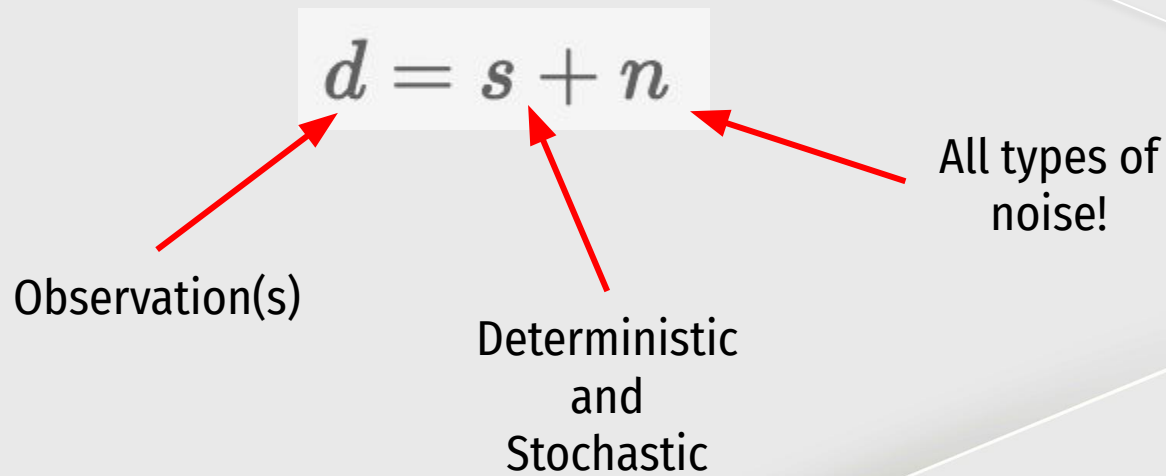



```
1  a = 1.3
2  floof_uncertainty = 1
3
4  callisto_floof = 3.1
5  callisto_uncertainty = 0.5
6
7  callisto_dist = np.random.normal(callisto_floof, callisto_uncertainty, int(1e6))
8  exact_relation = a*callisto_dist
9  relation_w_uncertainty = exact_relation + \
10 |     np.random.normal(0, floof_uncertainty, size=int(1e6))
11
12  final_mean = np.mean(relation_w_uncertainty)
13  final_std = np.std(relation_w_uncertainty)
14
```



Now to data analysis

But what *is* data?



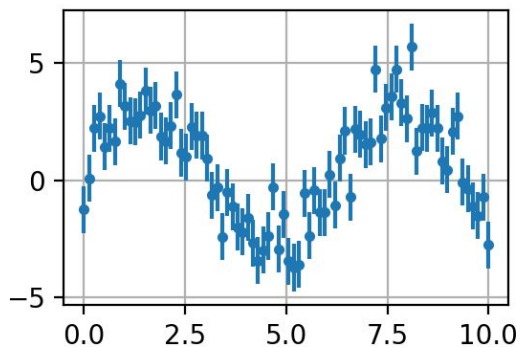
Signal Modeling

Move around the equation to make solutions easier!

- Subtracting noise is hard!

$$d = s + n$$

Data



Signal Modeling

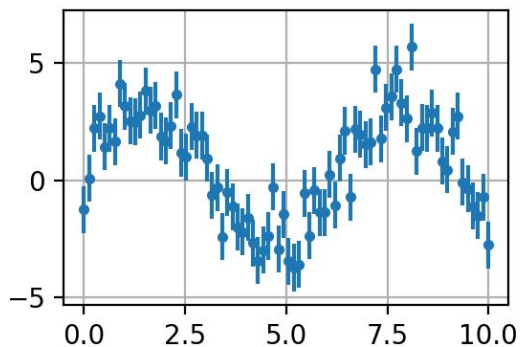
Move around the equation to make solutions easier!

- Subtracting noise is hard!
- Subtracting signals is easy!

$$d = s + n$$

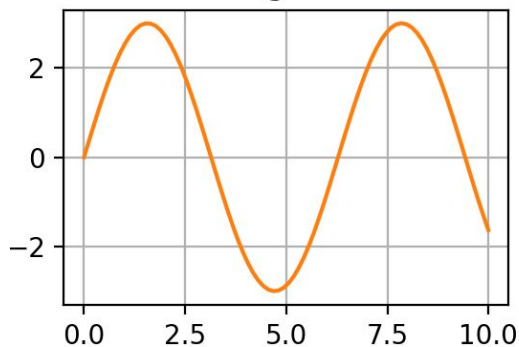
$$n = d - s$$

Data



-

Signal



=

Signal Modeling

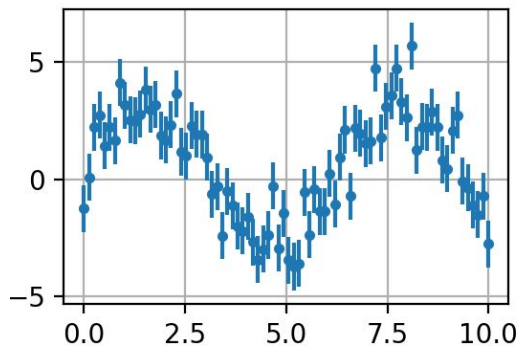
Move around the equation to make solutions easier!

- Subtracting noise is hard!
- Subtracting signals is easy!

$$d = s + n$$

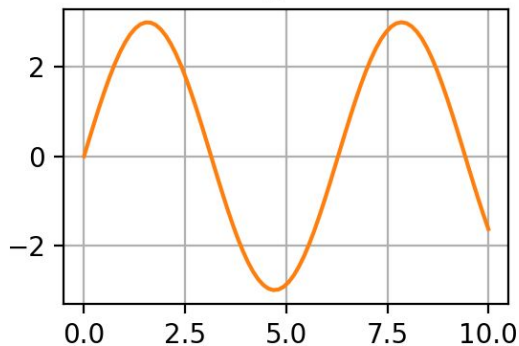
$$n = d - s$$

Data



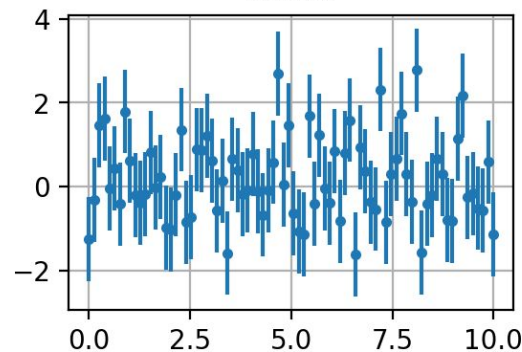
-

Signal



=

Noise



So Noise Modeling?

Testing for noise is easy! -> Gaussian likelihood

- A test for gaussianity

$$N(n|\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{n}{\sigma} \right)^2 \right]$$

$$d = s + n$$

$$n = d - s$$

Or, if variance is constant, log-likelihood is proportional to chi-square:

$$\ln N(n|\sigma) \propto \chi^2 = \left(\frac{n}{\sigma} \right)^2$$

Noise Modeling!

Subtract signal then maximize likelihood!

- Minimize chi-square

$$r_i = d_i - M(\theta)_i$$

$$\chi^2 = \sum_i \left(\frac{(r_i - M(\theta)_i)}{\sigma_i} \right)^2$$

Signal subtraction

Deterministic signals?

- Subtract from data

Stochastic signals?

- Model in covariance matrix

**This works as long as the residuals are
0-mean Gaussian random variables**

Noise weighted inner product of
residuals with residuals

$$r_i = d_i - M(\theta)_i$$

$$\chi^2 = \sum_i \left(\frac{(r_i - M(\theta)_i)}{\sigma_i} \right)^2$$

$$\chi^2 = \vec{r}^T \Sigma^{-1} \vec{r} = \langle r, r \rangle$$

$$\vec{r} = \vec{d} - M(\theta)$$

$$\Sigma = \langle \vec{r} \vec{r}^T \rangle$$

How do we find theta?

Frequentist approach:

Solve maximum-likelihood if possible!

- Analytic solution if linear model
- Numerical maximization if non-linear

Bayesian approach:

Use Markov Chain Monte Carlo (MCMC)

- Guess and check
- Map of the posterior space

$$\chi^2 = \vec{r}^T \Sigma^{-1} \vec{r}$$

$$\vec{r} = \vec{d} - M(\theta)$$

$$\Sigma = \langle \vec{r} \vec{r}^T \rangle$$

As an aside: Arianna Rosenbluth does not have her name on the algorithm yet made all of the code herself.

<https://www.aps.org/apsnews/2022/03/rosenbluth-metropolis-monte-carlo-algorithm>

Wait, MCMC?

Brute force grid searches work well in low dimensions

- Scaling is awful $O(n^{\text{dimension}})$

Hill-climbing algorithms are good for well-behaved functions

- Maximum likelihoods may be biased from numerical issues
- Stuck in local maximum

MCMCs fix all of this!

- Each point is dependent only on previous points (Markov chain)
- New points are chosen randomly around the parameter space (Monte Carlo)
- New guesses are judged based on Hastings ratio

$$H = \frac{p(d|\theta_{\text{proposed}})}{p(d|\theta_i)}$$

MCMC - An art and a science

MCMCs can be simple, or enormously complex

- Dataset and model dependent
- Tailor-made MCMCs > off-the-shelf MCMCs

Convergence time depends on proposals

- Where are good likelihood spots?
- “Cocktails” of proposal methods

End product must be post-processed

- Acceptance rate of new points (Depends on method)
 - Metropolis Hastings ~ (30-70%)
 - Gibbs sampling (100%)
- Stability (Gelman-Rubin statistic)
- Autocorrelation length (Number of independent samples)

A website to play with!

<https://chi-feng.github.io/mcmc-demo/app.html>

